Lucrarea nr. 5

1. Scopul lucrării

Lucrarea are ca scop studiul, respectiv proiectarea filtrelor FIR (*Finite Impulse Response*) cu ajutorul programelor MATLAB și dsPIC FD Lite. Se va realiza și implementarea filtrelor pe dsPIC33FJ256GP506.

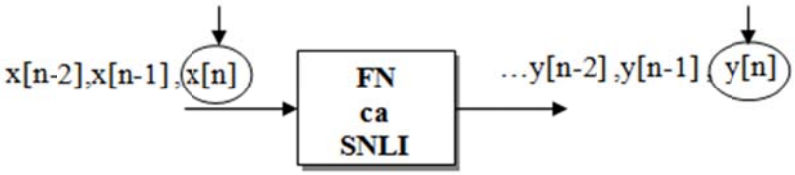
2. Noțiuni teoretice

Filtrul numeric este un sistem (în timp) discret caracterizat printr-un *algoritm numeric* cu ajutorul căruia o secvență de intrare se transformă într-o secvență (filtrată) de ieșire.

Algoritmul (de filtrare) poate să reprezinte: o filtrare trece jos, trece sus, trece bandă, oprește bandă, sau o operație de diferențiere, integrare etc.

2.1. Funcția de transfer a unui filtru numeric

Să considerăm schema generală de filtrare a unei secvențe de intrare $x[n]$ într-o secvență de ieșire $y[n]$:



Un FN poate fi caracterizat de ecuația în timp discret:

$$a_0 y[n] + a_1 y[n-1] + a_2 y[n-1] = b_0 x[n] + b_1 x[n-1] + b_2 x[n-2]$$

Dacă $a_0 = 1$, rezultă că secvența de ieșire este dată de relația:

$$y[n] = b_0 x[n] + b_1 x[n-1] + b_2 x[n-2] - a_1 y[n-1] - a_2 y[n-1]$$

sau:

$$y_n = b_0 x_n + b_1 x_{n-1} + b_2 x_{n-2} - a_1 y_{n-1} - a_2 y_{n-2}$$

Aplicând transformata *Z* ambilor membri rezultă că:

$$Y(z) = b_0 X(z) + b_1 z^{-1} X(z) + b_2 z^{-2} X(z) - a_1 z^{-1} Y(z) - a_2 z^{-2} Y(z)$$

Funcția de transfer a filtrului numeric considerat are expresia:

$$H(z) = \frac{Y(z)}{X(z)} = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}}$$

2.2. Avantajele filtrelor numerice

- flexibilitate prin modificarea algoritmului;
- ușor de proiectat, testat și implementat pe un microprocesor sau DSP;
- stabilitate ridicată în raport cu timpul și temperatura;
- siguranță în funcționare, dată de tehnologiile integrate;
- modularitate;
- versatilitate în prelucrarea semnalelor;
- aplicabilitate în joasă frecvență și radiofrecvență.

2.3. Etapele proiectării unui filtru numeric

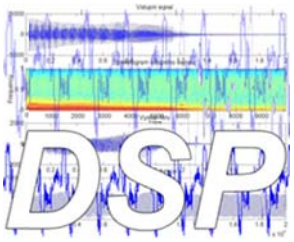
1. Rezolvarea problemei aproximării, care conduce la obținerea funcției de transfer $H\left(e^{j\omega}\right)$;
2. Alegerea schemei de realizare;
3. Cuantizarea coeficienților funcției $H(z)$, corespunzătoare unei lungimi finite de reprezentare;
4. Cuantizarea variabilelor: semnalul de intrare, de ieșire și a semnalelor intermediare;
5. Verificarea prin simulare (*offline*) a modului în care filtrul proiectat satisface exigențele impuse.
   De regulă, etapa 5 conduce la reluarea etapelor 2-4.

3. FN cu răspuns finit la impuls

Să considerăm schema generală a unui filtru numeric de tip FIR:







În timp discret, putem nota:

$$y[n] = b_0 x[n] + b_1 x[n-1] + \ldots + b_M x[n-M] = \sum_{i=0}^{M} b_i x[n-i]$$

Aplicând transformata *Z* ambilor membri ai ecuației de mai sus se obține:

$$Y(z) = \sum_{i=0}^{M} b_i z^{-i} X(z)$$

În planul variabilei *z*, funcția de transfer a FN de tip FIR este:

$$H_{FIR}(z) = \frac{Y(z)}{X(z)} = \frac{Z\{y[n]\}}{Z\{x[n]\}} = \sum_{i=0}^{M} b_i z^{-i}$$

iar răspunsul pondere este dat de relația:

$$h_{FIR}[n] = Z^{-1}\{H_{FIR}(z)\} = \sum_{i=0}^{M} b_i Z^{-1}\{z^{-i}\} = \sum_{i=0}^{M} b_i \delta[n-i] =$$
$$= b_0 \delta[n] + b_1 \delta[n-1] + \ldots + b_M \delta[n-M]$$

3.1. Proiectarea FN-FIR prin metoda ferestrelor (sau metoda seriei Fourier)

Algoritm:

1. Se dă răspunsul dorit pentru FN-FIR, notat cu $H_d(e^{j\omega})$, care este o funcție periodică în ω, cu perioada 2π;
2. Se determină răspunsul pondere al FN:

$$h_\infty[n] = TFTD^{-1}\{H_d(e^{j\omega})\} = \frac{1}{2\pi}\int_{2\pi} H_d(e^{j\omega}) \cdot e^{j\omega n} d\omega$$

3. Se trunchiază răspunsul pondere $h_\infty[n]$ pentru a obține răspunsul pondere al unui FN-FIR:

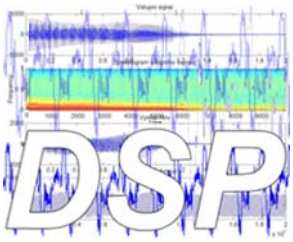
$$h[n] = h_\infty[n] \cdot w[n]$$

unde $w[n]$ este o funcție fereastră de lungime *N*.

4. Se calculează funcția de transfer a FN-FIR:

$$H(e^{j\omega}) = TFTD\{h[n]\} = TFTD\{h_\infty[n] \cdot w[n]\}$$

5. Se compară funcția de transfer obținută $H(e^{j\omega})$ cu funcția de transfer dorită $H_d(e^{j\omega})$.



3.2. Proiectarea FN-FIR prin metoda eșantionării în frecvență

În proiectarea filtrelor FIR prin metoda eșantionării în frecvență se specifică răspunsul în frecvență dorit $H_d(\omega)$ într-un set de *M* frecvențe uniform distanțate în intervalul $[0, 2\pi]$, fie acestea:

$$\omega_k = \frac{2\pi}{M}(k+\alpha), \quad k = 0,1,...(M-1)/2, \ M \ impar$$

$$k = 0,1,...\left[(M/2)-1\right], \quad M \ par$$

$$\alpha = \ 0 \ \ sau \ \ 1/2$$

și din aceste specificații se calculează răspunsul la impuls al filtrului sintetizat, care să îndeplinească cerințele de proiectare. Suportul teoretic al acestei metode îl constituie transformata Fourier discretă.

Se impune identitatea între răspunsul filtrului dorit și cel sintetizat la frecvențele specificate și se calculează răspunsul la impuls $h[n]$.

3.3. Proiectarea FN-FIR prin optimizare

Fie:

- caracteristica dorită a unui FN:

$$H_d\left(e^{j\omega}\right) = H_d(\omega)$$

- și caracteristica unui FN tip FIR:

$$H_{FIR}\left(e^{j\omega}\right) = H_{FIR}(\omega) = H_{FIR}(\omega, \bar{b})$$

unde $\bar{b}$ – coeficienți necunoscuți.

Pentru $\omega = \omega_i, i = \overline{1, M}$:

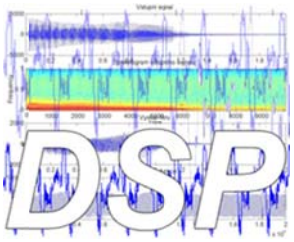
- *eroarea locală de aproximare* va fi:

$$\varepsilon(\omega_i, \bar{b}) = H_d(\omega_i) - H_{FIR}(\omega_i, \bar{b})$$

- *eroarea globală de aproximare* va fi:

$$E(\bar{b}) = \sum_{i=1}^{M} \varepsilon^2(\omega_i, \bar{b}) = \min$$

Dorim să minimizăm eroarea globală de aproximare prin alegerea corespunzătoare a coeficienților $\bar{b}$.



3.4. Aplicații FN-FIR în MATLAB

Comenzi MATLAB pentru proiectarea FN-FIR

Comandă	Descriere
fir1	metoda ferestrelor
fir2	metoda ferestrelor cu caracteristică de frecvență prescrisă
kaiserord	estimează parametrii pentru fir1 cu fereastra Kaiser
firls	utilizează metoda celor mai mici pătrate
remez	utilizează algoritmul Remez pentru proiectarea FIR
remezord	estimează ordinul FIR pentru utilizarea algoritmului Remez
fircls	utilizează metoda celor mai mici pătrate *cu constrângeri*
fircls1	utilizează metoda celor mai mici pătrate pentru FTJ și FTS
cremez	proiectează FIR cu fază neliniară echiriplu
firrcos	proiectează un FIR cu caracteristică tip cosinus ridicat

Proiectarea FN-FIR prin metoda ferestrelor

Exemplu de proiectare a unui FN-FIR cu $n = 51$ și $\omega t = 0,4 rad/s$ (se folosește o fereastră dreptunghiulară):

```
h=0.4*sinc (0.4*(-25:25));
subplot(211);
stem(h);grid
[H,w]=freqz (h,1,512,2);
subplot(212);
plot(w/2,abs (H)), grid
```

Exemple de proiectare a FN-FIR prin metoda ferestrelor

Se folosește funcția `b=fir1(n,wn,'ftype',window)` unde:
n+1 – reprezintă ordinul filtrului;
wn – frecvența de tăiere (are valori cuprinse între 0 și 1, unde 1 corespunde jumătății frecvenței de eșantionare – frecvența Nyquist);

'*ftype*' – tipul filtrului ('*high*', '*stop*');
window – tip fereastră.

Exemplu de FN-FIR de tipul 1 și 2

a) calculul răspunsurilor pondere și vizualizarea lor prin intermediul a trei tipuri de ferestre: *dreptunghiulară, Hamming, Blackman***:**

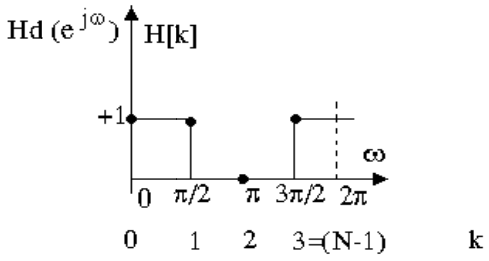
```
figure(1);
fd1=fir1(8,0.5,boxcar(9));fd2=fir1(9,0.5,boxcar(10));
subplot(321);stem(fd1);grid;subplot(322);stem(fd2);grid
;
fh1=fir1(8,0.5,hamming(9));fh2=fir1(9,0.5,hamming(10));
subplot(323);stem(fh1);grid;subplot(324);stem(fh2);grid
;
fb1=fir1(8,0.5,blackman(9));fb2=fir1(9,0.5,blackman(10)
);
subplot(325);stem(fb1);grid;subplot(326);stem(fb2);grid
;
```

b) calculul caracteristicilor de frecvență și vizualizarea lor:

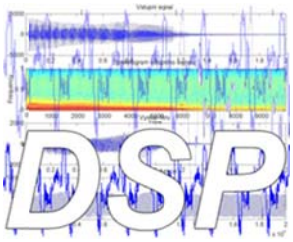
```
figure(2);
f=0:1/64:1-1/64;
Fd1=abs(fft(fd1,64)); Fd2=abs(fft(fd2,64));
subplot(321);plot(f,Fd1);grid;
subplot(322);plot(f,Fd2);grid;
Fh1=abs(fft(fh1,64));Fh2=abs(fft(fh2,64));
subplot(323);plot(f,Fh1);grid;
subplot(324);plot(f,Fh2);grid;
Fb1=abs(fft(fb1,64)); Fb2=abs(fft(fb2,64));
subplot(325);plot(f,Fb1);grid;
subplot(326);plot(f,Fb2);grid;
```

Proiectarea FN-FIR prin metoda eșantionării în frecvență

Dacă funcția de transfer dorită este $H[k]=[1 \quad 1 \quad 0 \quad 1]$:







funcţia de transfer este:

$$H(z)=\frac{1-z^{-N}}{N}\sum_{k=0}^{N-1}\frac{H[k]}{1-e^{jk\frac{2\pi}{N}}z^{-1}}$$

```
r=[1,1,0,1];N=4;
for k=0:N-1
    ka=k+1;p(ka)=exp (j*k*2*pi/N);K(ka)=0;
end
[b,a]=residuez (r,p,k);
bb (1:N)=b (1:N);bbb=[1,0,0,0,-1];
B=conv (bbb,bb);A=N.*a;
H=deconv (B,A);freqz (B,A);
```

Proiectarea unui FN-FIR tip trece jos cu $N=30$ cu caracteristica de frecvenţă prescrisă:

$$f=[0 \quad 0,6 \quad 0,6 \quad 1] \text{ şi } m=[1 \quad 1 \quad 0 \quad 0]$$

Se foloseşte funcţia **b = fir2(n, f, m)** unde:

n+1 – reprezintă ordinul filtrului;

f – este un vector cu valorile frecvenţei normate cu valorile între 0 şi 1, unde 1 corespunde frecvenţei Nyquist; prima componentă trebuie să fie 0, iar ultima 1.

m – este un vector cu valorile dorite ale amplitudinii funcţiei de transfer la frecvenţele normate corespunzătoare.

```
f=[0 0.6 0.6 1]; m=[1 1 0 0];
b=fir2(30,f,m);
subplot(211);stem(b);grid;
[h,w]=freqz(b,1,128);
subplot(212);plot(f/2,m,w/(2*pi),abs (h));grid;
```

Proiectarea unui FN-FIR tip trece sus prin metoda eşantionării în frecvenţă, plecând de la următoarele date de proiectare:

- ordinul $N=24$;
- frecvenţa limită de trecere $f_z=6kHz$;
- frecvenţa de eşantionare $f_e=20kHz$.

```
N=23;ft=6e3;fe=2e4;wn=ft/fe;inc=0.01;f=[0:inc:1];
Hd=[zeros(1,(2*wn)/inc+1)  ones(1,(1-2*wn)/inc)];
subplot(313);
```

```
plot(f/2,Hd);grid;
b=fir2(N,f,Hd);[H,W]=freqz (b,1,fe);
subplot (311);
plot(W/(2*pi), 20*log10 (abs(H)));grid;
subplot(312);
plot(W/(2*pi),unwrap(angle(H))*180/pi);grid;
```

Proiectarea FN-FIR tip cosinus ridicat

Se foloseşte funcţia **b=firrcos(n,F_0,df,Fs)**, unde:
n+1 – reprezintă ordinul filtrului;
F_0 – frecvenţa de tăiere [*Hz*];
df – banda de tranziţie [*Hz*];
Fs – frecvenţa de eşantionare.

Exemplu: un FN-FIR cosinus ridicat de ordinul $N = 20$ cu $f_0 = 0,25$ (frecvenţa de tăiere).

```
h=firrcos (20, 0.25,0.025,1);H=fft(h,1024);
plot((0:511)/1024,abs(H(1:512)));grid;
```

Proiectarea FN-FIR prin optimizare folosind algoritmul Remez elaborat de Parks-McClellan

Se foloseşte funcţia **b=remez(n,f,a,w)** unde:
n+1 – reprezintă ordinul filtrului;
f, a – caracteristica de frecvenţă impusă;
w – vector pondere;

Exemplu: proiectarea unui FTB numeric de gradul $N = 16$ specificat prin:

```
f=[0 0.3 0.4 0.6 0.7 1];
a=[0 0 1 1 0 0];
b=remez(16,f,a);
[h,w]=freqz (b,1,512);
plot (f,a,w/pi,abs(h));grid;
```

Proiectarea FN-FIR prin optimizare folosind minimizarea erorilor pătratice

Se foloseşte funcţia **b=firls(n,f,a,w,'ftype')** unde:
n+1 – reprezintă ordinul filtrului;

f, a – perechi de vectori care specifică caracteristica de frecvență prescrisă a filtrului;

w – vector pondere;

'ftype' – 'hilbbert', 'differentiator'.

Exemplu: se proiectează același filtru proiectat prin metoda remez și se compară rezultatele:

```
clear,clc;
f=[0 0.3 0.4 0.6 0.7 1];
a=[0 0 1 1 0 0];
b=firls(17,f,a);
[h,w]=freqz(b,1,512);
plot(f,a,w/pi,abs(h));grid;
```

Implementarea unui filtru FIR pe dsPIC33FJ256GP506

Relația în timp discret care caracterizează structura unui filtru FIR este:

$$y[n]=\sum_{i=0}^{N} x[n-i]\cdot h[i]$$

unde:

$y[n]$ – este semnalul de la ieșirea filtrului;

$x[n]$ – este semnalul de la intrarea filtrului;

$h[i]$ – sunt coeficienții filtrului;

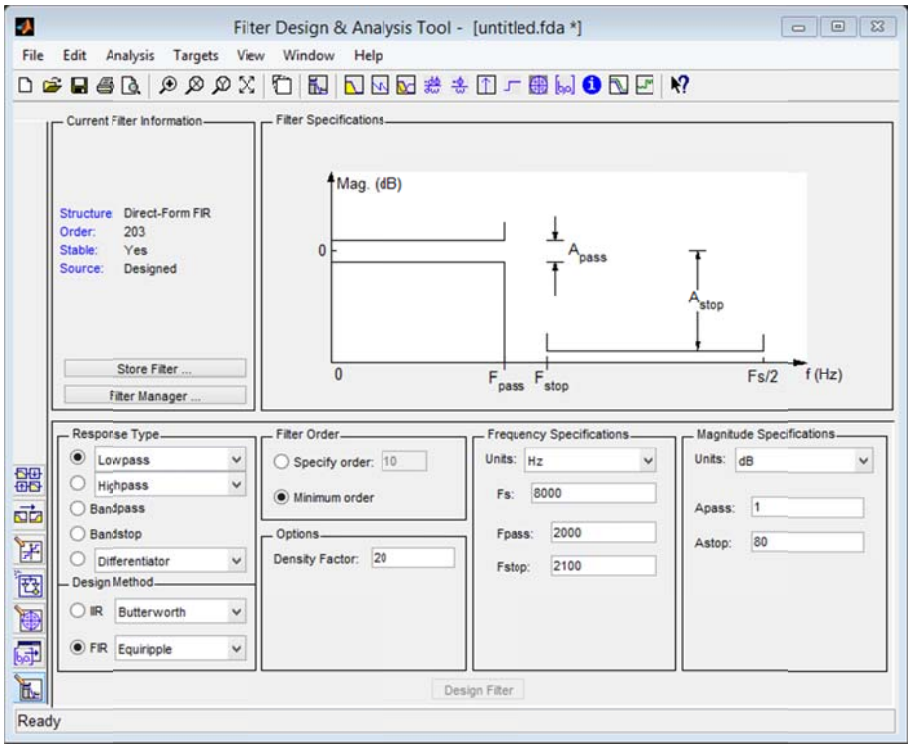
N – este ordinul filtrului.

Așa cum se poate observa, eșantionul semnalului de la ieșire la momentul de timp curent se calculează prin combinația liniară a eșantioanelor semnalului de la intrare. Deci, la fiecare moment de timp, trebuie calculată o sumă de produse, adică trebuie executate $N+1$ înmulțiri și N adunări, operanzii constând în istoria semnalului de la intrare pe de o parte și coeficienții filtrului pe de altă parte.

Se va implementa un filtru trece jos FIR, având frecvența de tăiere la $1kHz$. Pentru comparație, se va proiecta filtrul atât în MATLAB, cât și în dsPIC FD Lite.

Pentru a proiecta filtre în MATLAB, se folosește aplicația *Filter Design and Analysis*, care se poate accesa folosind comanda `>> fdatool`. Aceasta permite proiectarea filtrelor numerice, oferind o diversitate de opțiuni, cum ar fi: alegerea tipului filtrului, a ordinului său, stabilirea frecvențelor limită, a structurii filtrului, a modului de generare a coeficienților ș.a. De asemenea, prezintă grafic răspunsul filtrului proiectat, dar și polii/zerourile funcției de transfer.

După deschiderea aplicației, se realizează configurările din figură:



Înainte de a apăsa pe butonul *Design Filter*, se accesează *Set quantization parameters*, de pe coloana din stânga, și se alege *Filter Arithmetic – Fixed-point*. După ce se dă comanda *Design Filter*, se va putea observa răspunsul filtrului în funcție de frecvență.

Fișierul de coeficienți se obține astfel: *File* → *Export* → *Export To – Coefficient File (ASCII)* → *Format – Hexadecimal* → *Export*, și se alege locația în care se salvează.

Următorul pas îl reprezintă generarea fișierului de coeficienți în formatul utilizat de funcția *FIR* a bibliotecii dedicată pentru aplicații de procesare digitală de semnal a MPLAB-ului. Se va folosi funcția disponibilă în anexă:

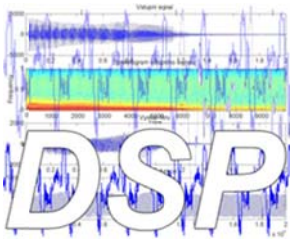
```
function [] = filter_file(filter_type,coeffs_File,memory)
```

Funcția are următorii parametri:

- *filter_type* – tipul filtrului proiectat (’*ftj*’, ’*fts*’, ’*ftb*’, ’*fob*’);
- *coeffs_file* – numele fișierului de coeficienți generat anterior (de exemplu, ’*untitled.fcf*’);
- *memory* – memoria în care vor fi stocați coeficienții (memoria RAM - ’*ram*’, sau memoria de program - ’*program*’).

Se va apela funcția astfel:

```
>> filter_file('ftj','untitled.fcf','program')
```



Se va genera fişierul *lowpass_psv.s*, care se va copia în proiectul nostru, în directorul *src*.

Proiectul va conţine fişierele antet *dsp.h* şi *dspcommon.inc*, şi fişierele sursă *fir.s*, *firdelay.s* şi cel furnizat de programul MATLAB. Fişierele *.s* se găsesc în *C:\Microchip Starter Kits\dsPIC Starter Kit - 1\MPLAB C30\src\dsp\asm*.

Rutinele scrise în limbaj de asamblare operează cu o structură numită *FIRStruct*, descrisă în *dsp.h*. Aceasta conţine următoarele variabile:

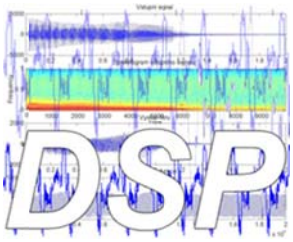
- *numCoeffs* – numărul de coeficienţi ai filtrului proiectat;
- *coeffsBase* – pointer către vectorul de coeficienţi;
- *coeffsEnd* – pointer către ultimul element din vectorul de coeficienţi;
- *coeffsPage* - variabilă care indică locaţia în care sunt stocaţi coeficienţii filtrului, în memoria de date *X* (caz în care, se introduce constanta COEFS_IN_DATA, definită în *dsp.h*) sau în memoria de program (în acest caz se introduce numărul paginii la care se află aceştia);
- *delayBase* – pointer către buffer-ul de întârziere;
- *delayEnd* – pointer către ultimul element din buffer-ul de întârziere;
- *delay* – valoarea curentă a pointer-ului de întârziere.

În procesul de filtrare trebuie să se ţină cont de eşantioanele filtrate şi cele care urmează să fie filtrate. Acest lucru se realizează folosind o secvenţă adiţională, numită *delay* (întârziere). Înainte de filtrare, secvenţa respectivă conţine eşantioanele unei secvenţe filtrate anterior. După operaţia de filtrare, secvenţa conţine ultimele eşantioane filtrate. Pentru operare corectă, trebuie ca această secvenţă să fie iniţializată cu 0. Acest lucru se realizează folosind funcţia *FIRDelayInit*, care acceptă ca argument pointer-ul către structura FIR definită cu funcţia *FIRStruct*.

Funcţia care realizează efectiv filtrarea este *FIR*, şi are următoarele argumente:

- *numSamps* – numărul de eşantioane de intrare (care trebuie filtrate);
- *dstSamps* – pointer către vectorul de ieşire, ce va conţine eşantioanele semnalului rezultat în urma operaţiei de filtrare;
- *srcSamps* – pointer către vectorul de intrare, ce conţine eşantioanele semnalului supus filtrării;
- *filter* – pointer către structura FIR definită cu funcţia *FIRStruct*.

Pentru această aplicaţie se va filtra un semnal dreptunghiular, de frecvenţă 1*kHz*, eşantionat cu frecvenţa de 20*kHz*, generat folosind programul dsPICworks. Pentru a nu exista probleme la compilare, se adaugă în arborele de căutare locaţia *D:\Projects\h*, iar în arborele de căutare pentru fişiere scrise în limbaj de asamblare (*Assembler Include Search Path*), locaţia *D:\Projects\src*.



Programul principal:

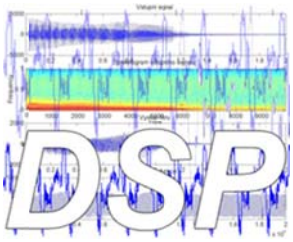
```
#include "..\h\p33FJ256GP506.h"
#include "..\h\dsp.h"

_FGS(GWRP_OFF & GCP_OFF); //Fara protectie la scriere si fara
protectia codului
_FOSCSEL(FNOSC_FRC); //Se foloseste oscilatorul RC rapid
(Fast RC)
_FOSC(FCKSM_CSECMD & OSCIOFNC_ON & POSCMD_NONE); //Se
activeaza comutarea ceasurilor, OSC2 ca pin digital, se
dezactiveaza oscilatorul primar
_FWDT(FWDTEN_OFF); //Se opreste cronometrul watchdog

#define BLOCK_LENGTH 256 //Marimea secventei care va fi
filtrata
#define LOWPASSFILTER
//#define HIGHPASSFILTER
//#define BANDPASSFILTER
//#define BANDSTOPFILTER
#define FILTERCOEFFS_IN_PROGMEM //Se comenteaza aceasta linie
daca coeficientii filtrului sunt stocati in memoria RAM

//Secventa de intrare - semnal dreptunghiular, de frecventa
1kHz, esantionat cu frecventa de 20kHz
fractional square1k[256] =
{0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
```



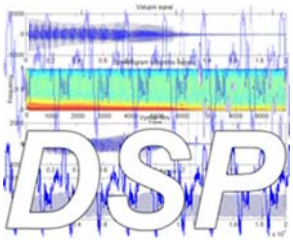



```
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001
};

//Secventa de iesire (secventa de intrare filtrata)
fractional FilterOut[BLOCK_LENGTH];

#ifdef LOWPASSFILTER
        #ifdef FILTERCOEFFS_IN_PROGMEM
                extern FIRStruct lowpass_psvFilter;
//Structura de filtru trece jos, cu coeficientii salvati in
memoria de program
        #else
                extern FIRStruct lowpassFilter;  //Structura
de filtru trece jos, cu coeficientii salvati in memoria RAM
        #endif
#endif
#ifdef HIGHPASSFILTER
        #ifdef FILTERCOEFFS_IN_PROGMEM
                extern FIRStruct highpass_psvFilter;
//Structura de filtru trece sus, cu coeficientii salvati in
memoria de program
        #else
                extern FIRStruct highpassFilter; //Structura
de filtru trece sus, cu coeficientii salvati in memoria RAM
        #endif
#endif
#ifdef BANDPASSFILTER
        #ifdef FILTERCOEFFS_IN_PROGMEM
                extern FIRStruct bandpassexample_psvFilter;
//Structura de filtru trece banda, cu coeficientii salvati in
memoria de program
        #else
                extern FIRStruct bandpassexampleFilter;
//Structura de filtru trece banda, cu coeficientii salvati in
memoria RAM
        #endif
#endif
#ifdef BANDSTOPFILTER
        #ifdef FILTERCOEFFS_IN_PROGMEM
```






```
                extern FIRStruct bandstop_psvFilter;
//Structura de filtru opreste banda, cu coeficientii salvati
in memoria de program
        #else
                extern FIRStruct bandstopFilter; //Structura
de filtru opreste banda, cu coeficientii salvati in memoria
RAM
        #endif
#endif

int main(void) {

                //Configurarea oscilatorului astfel incat
dispozitivul sa opereze la 40MHz
                //Fosc= Fin*M/(N1*N2), Fcy=Fosc/2
                //Fosc= 7.37M*40/(2*2)=80Mhz pentru tact
intrare de 7.37M
                PLLFBD=41;                      //M=39
                CLKDIVbits.PLLPOST=0;           //N1=2
                CLKDIVbits.PLLPRE=0;            //N2=2
                OSCTUN=0;

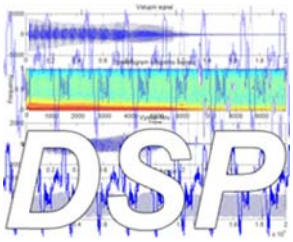
                __builtin_write_OSCCONH(0x01);
        //Initializare oscilator la FRC (Fast RC) cu PLL
                __builtin_write_OSCCONL(0x01);
                while (OSCCONbits.COSC != 0b01);
        //Astepta realizarea comutarii
                while(!OSCCONbits.LOCK);

         //Realizarea filtrarii
         #ifdef LOWPASSFILTER
                 #ifdef FILTERCOEFFS_IN_PROGMEM
                 //Initializarea secventei delay
                         FIRDelayInit(&lowpass_psvFilter);
                 //Filtrarea secventei de intrare

FIR(BLOCK_LENGTH,&FilterOut[0],&square1k[0],&lowpass_psvFilte
r);
                 #else
                 //Initializarea secventei delay
                         FIRDelayInit(&lowpassexampleFilter);
                 //Filtrarea secventei de intrare

FIR(BLOCK_LENGTH,&FilterOut[0],&square1k[0],&lowpassFilter);
                 #endif
                #endif
                #ifdef HIGHPASSFILTER
                 #ifdef FILTERCOEFFS_IN_PROGMEM
                 //Initializarea secventei delay
```






```
                        FIRDelayInit(&highpass_psvFilter);
                //Filtrarea secventei de intrare

FIR(BLOCK_LENGTH,&FilterOut[0],&square1k[0],&highpass_psvFilt
er);
                #else
                //Initializarea secventei delay
                        FIRDelayInit(&highpassFilter);
                //Filtrarea secventei de intrare

FIR(BLOCK_LENGTH,&FilterOut[0],&square1k[0],&highpassFilter);
                #endif
               #endif
        #ifdef BANDPASSFILTER
                #ifdef FILTERCOEFFS_IN_PROGMEM
                //Initializarea secventei delay
                        FIRDelayInit(&bandpass_psvFilter);
                //Filtrarea secventei de intrare

FIR(BLOCK_LENGTH,&FilterOut[0],&square1k[0],&bandpass_psvFilt
er);
                #else
                //Initializarea secventei delay
                        FIRDelayInit(&bandpassFilter);
                               //Filtrarea secventei de
intrare

FIR(BLOCK_LENGTH,&FilterOut[0],&square1k[0],&bandpassFilter);
                #endif
               #endif
               #ifdef BANDSTOPFILTER
                #ifdef FILTERCOEFFS_IN_PROGMEM
                //Initializarea secventei delay
                        FIRDelayInit(&bandstop_psvFilter);
                //Filtrarea secventei de intrare

FIR(BLOCK_LENGTH,&FilterOut[0],&square1k[0],&bandstop_psvFilt
er);
                #else
                //Initializarea secventei delay
                        FIRDelayInit(&bandstopFilter);
                //Filtrarea secventei de intrare

FIR(BLOCK_LENGTH,&FilterOut[0],&square1k[0],&bandstopFilter);
                #endif
        #endif

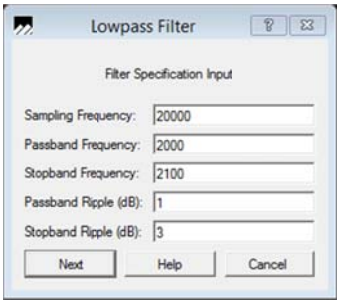
        //Bucla de program
        while (1);}
```

Se vor vizualiza secvențele de intrare și de ieșire.

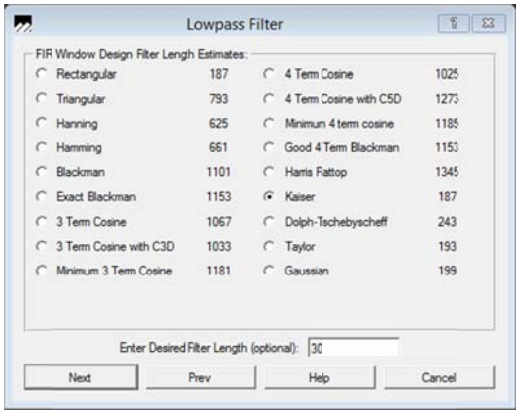
Programul principal este conceput utilizând directive de preprocesare, astfel încât să permită folosirea tuturor tipurilor de filtre. De exemplu, dacă se dorește utilizarea unui filtru trece bandă, atunci se urmează pașii:

- se proiectează filtrul în MATLAB sau dsPIC FD Lite și se generează fișierul de coeficienți în forma necesară funcțiilor din MPLAB;
- se adaugă fișierul respectiv proiectului;
- se comentează liniile *#define LOWPASSFILTER*, *#define HIGHPASSFILTER* și *#define BANDSTOPFILTER*;
- se scoate de sub comentariu linia *#define BANDPASSFILTER*;
- dacă se dorește salvarea coeficienților în memoria RAM, atunci se comentează și linia *#define FILTERCOEFFS_IN_PROGMEM*.

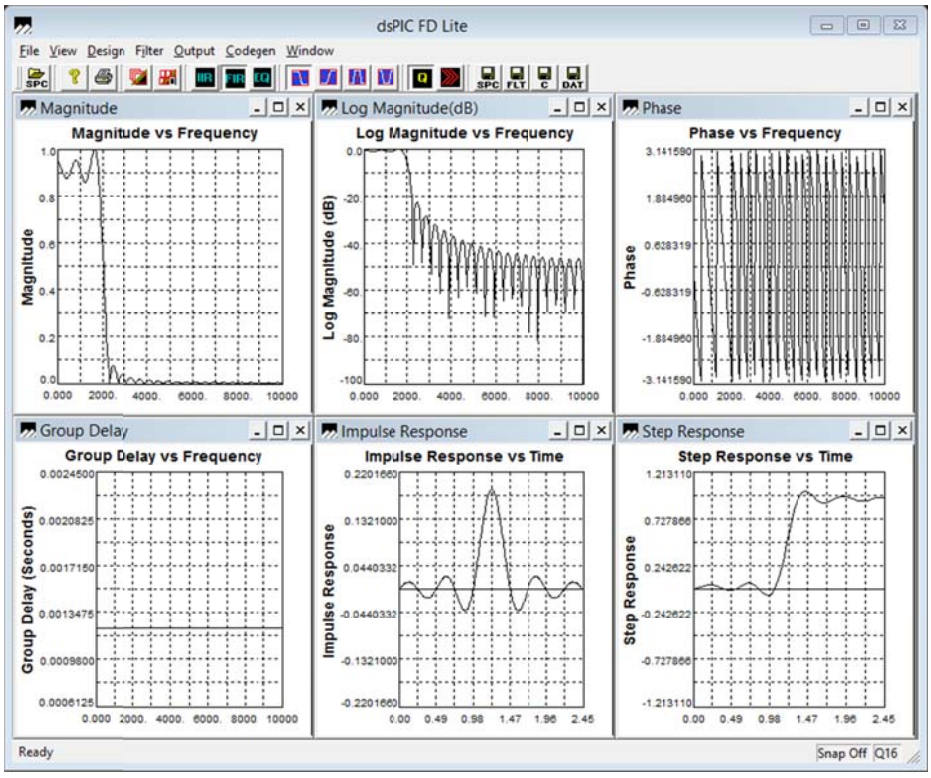
În continuare este prezentată procedura de proiectare a unui filtru folosind programul dsPIC FD Lite. După deschidere, se urmează secvența: *Design → FIR Window Design → Lowpass* și se introduc datele, ca în figura de mai jos:



Se alege tipul ferestrei și/sau se poate specifica ordinul filtrului. Am ales ordinul filtrului 30.

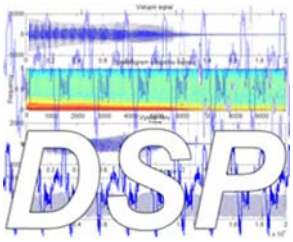


Următoarea fereastră prezintă grafic răspunsul filtrului și variația altor parametri în funcție de frecvență:



Pentru generarea fișierului cu coeficienți se selectează: *Codegen* → *Microchip* → *dsPIC30* → *Program Space* → se debifează *C Header File and Sample Calling Sequence (.h)* și *C Wrapper*. Se va salva fișierul cu denumirea *lowpass_psv*. Dacă coeficienții ar fi fost stocați în RAM, se aleagea denumirea *lowpass*. În mod asemănător, se procedează și pentru alte tipuri de filtre, cu mențiunea că se folosesc denumirile *highpass*, *passband*, respectiv *stopband*.

Fișierul generat de programul dsPIC FD Lite se adaugă proiectului, la fel cum s-a procedat în cazul proiectării folosind MATLAB.



4. Desfășurarea lucrării

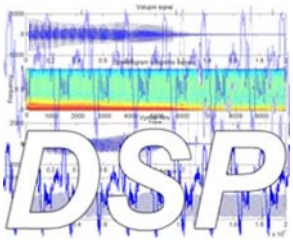
1. Se vor rula în MATLAB toate aplicațiile descrise în breviarul teoretic și se va urmări execuția lor.

2. Se va implementa pe dsPIC33FJ256GP506 un FTB cu banda $[800,1000]Hz$, frecvența de eșantionare $8kHz$, gradul 511, folosind metoda ferestrelor.

3. Să se implementeze pe procesorul dsPIC33FJ256GP506 un FTB cu banda $[2000,3000]Hz$, frecvența de eșantionare $32kHz$, gradul 399, folosind metoda ferestrelor.

4. Să se proiecteze un FTJ cu frecvența de tăiere $2000Hz$, frecvența de eșantionare $18,9kHz$, gradul 300, folosind metoda eșantionării în frecvență. Să se implementeze pe dsPIC33FJ256GP506.

5. Să se proiecteze un FTS cu frecvența de tăiere $10kHz$, frecvența de eșantionare $48kHz$, gradul 250, prin optimizare folosind algoritmul *Remez*.



Anexă

Programul MATLAB folosit pentru generarea fișierului de coeficienți

```
function [] =
filter_file(filter_type,coeffs_File,memory)

%fdatool
switch memory
    case 'program'
        switch filter_type
            case 'ftj'
                file = 'lowpass_psv.s';
            case 'fts'
                file = 'highpass_psv.s';
            case 'ftb'
                file = 'bandpass_psv.s';
            case 'fob'
                file = 'bandstop_psv.s';
        end

        fileID = fopen(coeffs_File);
        string = fscanf(fileID,'%s');

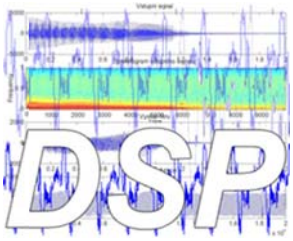
        k = strfind(string,'FilterLength:');
        k1 = strfind(string,'%Stable');
        filter_order = str2double(string(k+13:k1-
1));

        k = strfind(string,'Numerator:');
        j = 1;
        i = k(length(k)) + 10;
        while (i <= length(string))
            coef(j,:) = string(i:i+3);
            j = j + 1;
            i = i + 4;
        end

        fileID = fopen(file,'w');

        string = {';
.................................................
.............................\r\n'
```





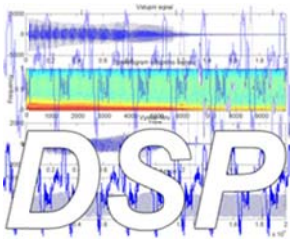

```
        horzcat(';    File
',file(1:length(file)-2),'.s\r\n')
        ';
.....................................................
.........................\r\n'
        '\r\n'
        horzcat('                .equ
',file(1:length(file)-2),'NumTaps,
',num2str(filter_order))
        '\r\n'
        ';
.....................................................
.........................\r\n'
        '; Allocate and initialize filter
taps\r\n'
        horzcat(';               .section
.',file(1:length(file)-2),'const, "x"  ;<-Syntax
supported in MPLAB C30\r\n')
        '
;v1.20 and before\r\n'
        horzcat('                .section
.',file(1:length(file)-2),'const, code ;<-Syntax
supported in MPLAB C30\r\n')
        '
;v1.30 and later\r\n'
        horzcat('                .align
',num2str(2^nextpow2(filter_order*2)),'\r\n')
        '\r\n'
        horzcat(file(1:length(file)-2),'Taps:
\r\n')};

        for row = 1:length(string)
            fprintf(fileID,string{row});
        end

        for i = 1:length(coef)
            fprintf(fileID,horzcat('.hword
0x',coef(i,:),'\r\n'));
        end

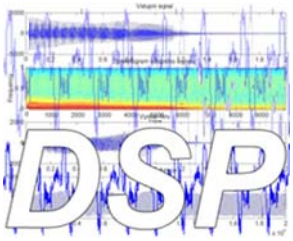
        string = {'\r\n'
        ';
.....................................................
.........................\r\n'
```





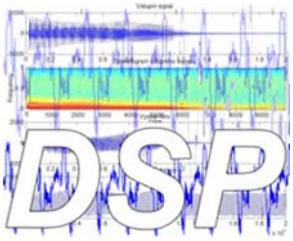

```
        '; Allocate delay line in (uninitialized)
Y data space\r\n'
        '\r\n'
        ';              .section .ybss,  "b"
;<-Syntax supported in MPLAB C30\r\n'
        '
;v1.20 and before\r\n'
        '               .section .ydata, data,
ymemory  ;<-Syntax supported in MPLAB C30\r\n'
        '
;v1.30 and later\r\n'
        horzcat('               .align
',num2str(2^nextpow2(filter_order*2)),'\r\n')
        '\r\n'
        horzcat(file(1:length(file)-
2),'Delay:\r\n')
        horzcat('               .space
',file(1:length(file)-2),'NumTaps*2\r\n')
        '\r\n'
        ';
.................................................
.............................\r\n'
        '; Allocate and intialize filter
structure\r\n'
        '\r\n'
        '               .section .data\r\n'
        horzcat('               .global
_',file(1:length(file)-2),'Filter\r\n')
        '\r\n'
        horzcat('_',file(1:length(file)-
2),'Filter:\r\n')
        horzcat('.hword ',file(1:length(file)-
2),'NumTaps\r\n')
        horzcat('.hword
psvoffset(',file(1:length(file)-2),'Taps)\r\n')
        horzcat('.hword
psvoffset(',file(1:length(file)-
2),'Taps)+',file(1:length(file)-2),'NumTaps*2-
1\r\n')
        horzcat('.hword
psvpage(',file(1:length(file)-2),'Taps)\r\n')
        horzcat('.hword ',file(1:length(file)-
2),'Delay\r\n')
```






```
        horzcat('.hword ',file(1:length(file)-
2),'Delay+',file(1:length(file)-2),'NumTaps*2-
1\r\n')
        horzcat('.hword ',file(1:length(file)-
2),'Delay\r\n')
        '\r\n'
        '; 
.................................................
.........................\r\n'
        '; 
.................................................
.........................\r\n'
        '; Sample assembly language calling
program\r\n'
        ';  The following declarations can be cut
and pasted as needed into a program\r\n'
        ';              .extern
_FIRFilterInit\r\n'
        ';              .extern
_BlockFIRFilter\r\n'
        horzcat(';              .extern
_',file(1:length(file)-2),'Filter\r\n')
        ';\r\n'
        ';              .section       .bss\r\n'
        ';\r\n'
        ';       The input and output buffers can
be made any desired size\r\n'
        ';          the value 40 is just an
example - however, one must ensure\r\n'
        ';          that the output buffer is at
least as long as the number of samples\r\n'
        ';          to be filtered (parameter
4)\r\n'
        ';input:         .space  40\r\n'
        ';output:        .space  40\r\n'
        ';              .text\r\n'
        ';\r\n'
        ';\r\n'
        ';  This code can be copied and pasted as
needed into a program\r\n'
        ';\r\n'
        ';\r\n'
        '; Set up pointers to access input
samples, filter taps, delay line and\r\n'
```






```
        '; output samples.\r\n'
        horzcat(';                mov
#_',file(1:length(file)-2),'Filter, W0  ;
Initalize W0 to filter structure\r\n')
        ';                call    _FIRFilterInit  ;
call this function once\r\n'
        ';\r\n'
        '; The next 4 instructions are required
prior to each subroutine call\r\n'
        '; to _BlockFIRFilter\r\n'
        horzcat(';                mov
#_',file(1:length(file)-2),'Filter, W0  ;
Initalize W0 to filter structure\r\n')
        ';                mov     #input, W1      ;
Initalize W1 to input buffer\r\n'
        ';                mov     #output, W2     ;
Initalize W2 to output buffer\r\n'
        ';                mov     #20, W3 ;
Initialize W3 with number of required output
samples\r\n'
        ';                call    _BlockFIRFilter ;
call as many times as needed\r\n'
        };

        for row = 1:length(string)
            fprintf(fileID,string{row});
        end

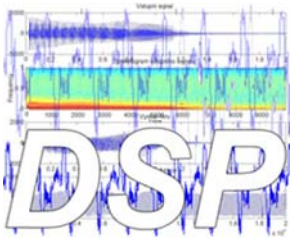
        fclose(fileID);

    case 'ram'

        switch filter_type
            case 'ftj'
                file = 'lowpass.s';
            case 'fts'
                file = 'highpass.s';
            case 'ftb'
                file = 'bandpass.s';
            case 'fob'
                file = 'bandstop.s';
        end

        fileID = fopen(coeffs_File);
```






```
        string = fscanf(fileID,'%s');

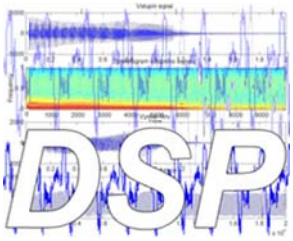
        k = strfind(string,'FilterLength:');
        k1 = strfind(string,'%Stable');
        filter_order = str2double(string(k+13:k1-
1));

        k = strfind(string,'Numerator:');
        j = 1;
        i = k(length(k)) + 10;
        while (i <= length(string))
            coef(j,:) = string(i:i+3);
            j = j + 1;
            i = i + 4;
        end

        fileID = fopen(file,'w');

        string = {';
.................................................
.............................\r\n'
        horzcat(';    File
',file(1:length(file)-2),'.s\r\n')
        ';
.................................................
.............................\r\n'
        '\r\n'
        horzcat('                .equ
',file(1:length(file)-2),'NumTaps,
',num2str(filter_order))
        '\r\n'
        ';
.................................................
.............................\r\n'
        '; Allocate and initialize filter
taps\r\n'
        horzcat(';               .section
.',file(1:length(file)-2),'.xdata
;<-Syntax supported in MPLAB C30\r\n')
        '
;v1.20 and before\r\n'
        horzcat('                .section
.',file(1:length(file)-2),'.xdata, data, xmemory
;<-Syntax supported in MPLAB C30\r\n')
```





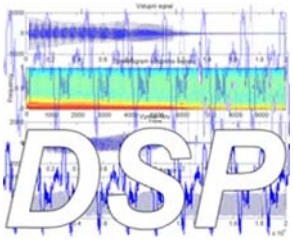

```
        '
;v1.30 and later\r\n'
        horzcat('                .align
',num2str(2^nextpow2(filter_order*2)),'\r\n')
        '\r\n'
        horzcat(file(1:length(file)-2),'Taps:
\r\n')};

        for row = 1:length(string)
            fprintf(fileID,string{row});
        end

        for i = 1:length(coef)
            fprintf(fileID,horzcat('.hword
0x',coef(i,:),'\r\n'));
        end

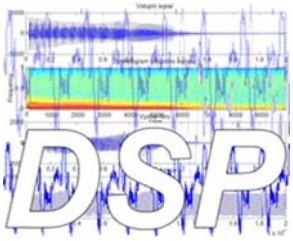
        string = {'\r\n'
        ';
.................................................
.............................\r\n'
        '; Allocate delay line in (uninitialized)
Y data space\r\n'
        '\r\n'
        ';               .section .ybss,  "b"
;<-Syntax supported in MPLAB C30\r\n'
        '
;v1.20 and before\r\n'
        '                .section .ydata, data,
ymemory  ;<-Syntax supported in MPLAB C30\r\n'
        '
;v1.30 and later\r\n'
        horzcat('                .align
',num2str(2^nextpow2(filter_order*2)),'\r\n')
        '\r\n'
        horzcat(file(1:length(file)-
2),'Delay:\r\n')
        horzcat('                .space
',file(1:length(file)-2),'NumTaps*2\r\n')
        '\r\n'
        ';
.................................................
.............................\r\n'
```





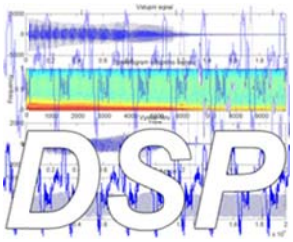

```
        '; Allocate and intialize filter
structure\r\n'
        '\r\n'
        '                .section .data\r\n'
        horzcat('                .global
_',file(1:length(file)-2),'Filter\r\n')
        '\r\n'
        horzcat('_',file(1:length(file)-
2),'Filter:\r\n')
        horzcat('.hword ',file(1:length(file)-
2),'NumTaps\r\n')
        horzcat('.hword
psvoffset(',file(1:length(file)-2),'Taps)\r\n')
        horzcat('.hword
psvoffset(',file(1:length(file)-
2),'Taps)+',file(1:length(file)-2),'NumTaps*2-
1\r\n')
        '.hword 0xff00\r\n'
        horzcat('.hword ',file(1:length(file)-
2),'Delay\r\n')
        horzcat('.hword ',file(1:length(file)-
2),'Delay+',file(1:length(file)-2),'NumTaps*2-
1\r\n')
        horzcat('.hword ',file(1:length(file)-
2),'Delay\r\n')
        '\r\n'
        '
.................................................
.........................\r\n'
        '
.................................................
.........................\r\n'
        '; Sample assembly language calling
program\r\n'
        ';  The following declarations can be cut
and pasted as needed into a program\r\n'
        ';               .extern
_FIRFilterInit\r\n'
        ';               .extern
_BlockFIRFilter\r\n'
        horzcat(';               .extern
_',file(1:length(file)-2),'Filter\r\n')
        ';\r\n'
        ';               .section        .bss\r\n'
```






```
        ';\r\n'
        ';        The input and output buffers can
be made any desired size\r\n'
        ';          the value 40 is just an
example - however, one must ensure\r\n'
        ';          that the output buffer is at
least as long as the number of samples\r\n'
        ';          to be filtered (parameter
4)\r\n'
        ';input:         .space  40\r\n'
        ';output:        .space  40\r\n'
        ';               .text\r\n'
        ';\r\n'
        ';\r\n'
        ';  This code can be copied and pasted as
needed into a program\r\n'
        ';\r\n'
        ';\r\n'
        '; Set up pointers to access input
samples, filter taps, delay line and\r\n'
        '; output samples.\r\n'
        horzcat(';              mov
#_',file(1:length(file)-2),'Filter, W0  ;
Initalize W0 to filter structure\r\n')
        ';              call    _FIRFilterInit  ;
call this function once\r\n'
        ';\r\n'
        '; The next 4 instructions are required
prior to each subroutine call\r\n'
        '; to _BlockFIRFilter\r\n'
        horzcat(';              mov
#_',file(1:length(file)-2),'Filter, W0  ;
Initalize W0 to filter structure\r\n')
        ';              mov     #input, W1      ;
Initalize W1 to input buffer\r\n'
        ';              mov     #output, W2     ;
Initalize W2 to output buffer\r\n'
        ';              mov     #20, W3 ;
Initialize W3 with number of required output
samples\r\n'
        ';              call    _BlockFIRFilter ;
call as many times as needed\r\n'
        };
```






```
            for row = 1:length(string)
                fprintf(fileID,string{row});
            end

            fclose(fileID);
    end
```